

1 React `useCallback` Hook – Complete Guide (Beginner → Advanced)

2 What is useCallback?

React ma component re-render huda, component vitra declare gareko **function** haru pani feri create hunxan.

Sadharan project ma yo problem hudaina.

Tara thulo application ma, same function harek render ma recreate huda unnecessary child component re-render huna sakxa, jase performance ghatauna sakxa.

Yei problem solve garna React le **useCallback** hook provide gareko xa.

3 Official Definition

useCallback is a React Hook that memoizes a function and returns the same function reference until one of its dependencies changes.

Simple Nepali-English ma:

useCallback le function lai memory ma save garxa ra dependency change nahunjel samma tei function reuse garxa.

4 Why do we need useCallback?

React ma component re-render huda function haru feri create hunxan.

Example

```
function App() {  
  const [count, setCount] = useState(0);  
  const handleClick = () => {
```

```

    console.log("Hello");
  };

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  );
}

```

Question:

Button click garepaxi ke hunxa?

Component Render

↓

handleClick Function Create

↓

Component Render

↓

New handleClick Function Create

↓

Again New Function

Function ko code same bhaye pani React ko lagi yo **new function reference** ho.

5 JavaScript Function Reference

Function pani object ho.

```

const a = () => {};
const b = () => {};

console.log(a === b);

```

Output

false

Why?

Because

Memory

a → Address A

b → Address B

Duita function ko code same xa.

Tara memory address farak xa.

React le yei reference compare garxa.

6 Parent & Child Problem

Suppose

App

↓

Child

Parent Component

```
import Child from "../Child";
import { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  const handleClick = () => {
    console.log("Clicked");
  };

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <Child onClick={handleClick} />
    </>
  );
}
```

```
export default App;
```

Child Component

```
function Child({ onClick }) {  
  
  console.log("Child Render");  
  
  return (  
    <button onClick={onClick}>  
      Child Button  
    </button>  
  );  
}  
  
export default Child;
```

7 What Happens?

Button click

Parent Render

↓

New handleClick Function

↓

Child receives New Prop

↓

Child Render Again

Even though Child ko data change bhayeko xaina.

8 Solution – useCallback

React lai bhanchau

"Yo function save gara."

"Dependency change nahunjel samma tei function use gara."

Syntax

```
const memoizedFunction = useCallback(() => {  
  }, [dependencies]);
```

9 Understanding the Syntax

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

Meaning

First Render

↓

Create Function

↓

Store Function

↓

Next Render

↓

Reuse Same Function

10 Dependency Array

11 Empty Dependency

```
useCallback(() => {  
  }, []);
```

Meaning

Function only once create hunxa.

12 One Dependency

```
const handleClick = useCallback(() => {
```

```
    console.log(count);  
  }, [count]);
```

Meaning

Count change

↓

Create New Function

Else

↓

Reuse Previous Function

13 Complete Example

```
import { useCallback, useState } from "react";  
  
function App() {  
  const [count, setCount] = useState(0);  
  
  const handleClick = useCallback(() => {  
    console.log("Button Clicked");  
  }, []);  
  
  return (  
    <>  
      <h1>{count}</h1>  
  
      <button onClick={() => setCount(count + 1)}>  
        Increase  
      </button>  
  
      <button onClick={handleClick}>  
        Click Me  
      </button>  
    </>  
  );  
}  
  
export default App;
```

Here,

Increase button click gare pani

handleClick

↓

Same Function Reference

14 Real Benefit

Real benefit tab matra dekhinxa jaba Child component lai `React.memo()` le wrap gareko hunxa.

Child

```
import React from "react";

const Child = React.memo(({ onClick }) => {

  console.log("Child Render");

  return (
    <button onClick={onClick}>
      Child
    </button>
  );

});

export default Child;
```

Now

Parent Render

↓

Same Function Reference

↓

React.memo compares Props

↓

Props Same

↓

Child Doesn't Re-render

15 Real World Example

Suppose

E-Commerce Website

100 Product Cards xa.

```
<ProductCard
  product={product}
  onAddToCart={handleAddToCart}
/>
```

Without useCallback

Parent Render

↓

New handleAddToCart

↓

100 Product Cards Render Again

Performance decrease.

Using useCallback

```
const handleAddToCart = useCallback((id) => {
  console.log(id);
}, []);
```

Now

Parent Render

↓

Same Function

↓

React.memo

↓

Cards Don't Re-render

Performance better.

16 Dependency Example

```
const handleClick = useCallback(() => {  
    console.log(count);  
}, [count]);
```

Flow

Count = 0

↓

Function Stored

↓

Count = 1

↓

Dependency Changed

↓

New Function Stored

17 Common Mistakes

18 Mistake 1

Wrong Dependency

```
const handleClick = useCallback(() => {  
    console.log(count);  
}, []);
```

Problem

Old Count Print Hunxa

Correct

```
const handleClick = useCallback(() => {
```

```
    console.log(count);  
  }, [count]);
```

19 Mistake 2

Using Everywhere

Wrong

```
const sayHi = useCallback(() => {  
    console.log("Hi");  
}, []);
```

Simple function lai memoize garnu unnecessary ho.

20 Mistake 3

Thinking useCallback Stops Re-render

No.

useCallback

↓

Same Function Reference

↓

NOT

↓

Stop Parent Render

Parent ta state change huda render hunai parxa.

21 When Should You Use useCallback?

- ☑ Passing function to `React.memo` child components.
 - ☑ Event handlers (`onClick`, `onSubmit`, `onChange`) shared with memoized children.
 - ☑ Stable callback required in dependency arrays of other hooks.
 - ☑ Large applications where unnecessary re-renders matter.
-

22 When NOT to Use `useCallback`?

- ✗ Every function.
- ✗ Every component.
- ✗ Small applications.
- ✗ Functions that are never passed as props.

Remember:

Optimization ko lagi optimization nagarne.

23 `useMemo` vs `useCallback`

Feature	<code>useMemo</code>	<code>useCallback</code>
Stores	Value	Function
Returns	Value	Function
Purpose	Avoid recalculating expensive values	Keep same function reference
Common Uses	Filter, Sort, Reduce, Heavy Calculation	Event Handlers
Prevents	Unnecessary calculations	Unnecessary child re-renders (with <code>React.memo</code>)

24 Interview Questions

25 1. What is useCallback?

`useCallback` is a React Hook that memoizes a function and returns the same function reference until its dependencies change.

26 2. Does useCallback stop component re-render?

No.

It only keeps the same function reference.

To reduce unnecessary child re-renders, it is commonly used together with `React.memo`.

27 3. Difference between useMemo and useCallback?

- `useMemo` remembers a **value**.
 - `useCallback` remembers a **function**.
-

28 Best Practices

- Use only when necessary.
 - Always specify correct dependencies.
 - Combine with `React.memo` for better optimization.
 - Don't wrap every function with `useCallback`.
 - Measure performance before optimizing.
-

29 Quick Revision

Feature	<code>useCallback</code>
Hook Type	Performance Optimization
Saves	Function
Returns	Memoized Function
Recreates Function	Only when dependency changes
Common Use	Event Handlers passed to child components

Feature	useCallback
Stops Parent Render?	☒ No
Works Best With	☒ <code>React.memo</code>

30 Easy Trick to Remember

`useMemo`

↓

Remembers VALUE

Example

Filtered Products
Sorted Products
Total Price
`useCallback`

↓

Remembers FUNCTION

Example

`handleClick()`
`handleSubmit()`
`handleDelete()`
`handleAddToCart()`

31 Final Summary

- `useCallback` is used to **memoize functions**.
- It prevents creating a **new function reference** on every render.
- It is most useful when passing functions to **memoized child components**.
- By itself, `useCallback` does **not** stop re-renders.
- For maximum benefit, use it together with **React.memo**.